

Dutch

Product Requirements & Technical Specifications Document

1. Executive Summary

Dutch is a comprehensive expense-splitting application designed to seamlessly manage shared costs among friends and groups. It features an offline-first architecture powered by event sourcing, ensuring reliable cross-device synchronization. Critical features include multi-currency support anchored to USD, granular expense splitting (contributions vs. owed), and a robust peer-to-peer payment approval workflow.

2. Core Features & Requirements

2.1 User & Network Management

- **Authentication:** Registration and login via mobile phone number (OTP).
- **Friend System:** Users can add friends by searching their registered phone numbers.
- **Implicit Connections:** Adding a user to a group automatically establishes a friend connection.

2.2 Group & Expense Management

- **Groups:** Create named groups, add friends, and maintain a running ledger of debts.
- **Expense Entry:** Capture Currency, Total Amount, Date, Time (optional), and Description.
- **Flexible Splitting:**
 - **Contributions (Who Paid):** Supports multiple payers per expense.
 - **Splits (Who Owes):** Supports exact amounts, percentages, or equal splits.
 - **Validation Rule:** The application must strictly enforce that Total Paid equals Total Split.

2.3 Multi-Currency & USD Anchoring

- All transactions (expenses and settlements) are recorded in their native currency.
- At the exact time of transaction, the app fetches the live FX rate and anchors the value to USD.
- All internal ledger calculations (who owes whom) are computed using the anchored USD amounts.
- UI presentation converts the USD balance back to the user's preferred local currency for readability.

2.4 Payment & Approval Workflow

- **Initiation:** User 1 records a payment sent to User 2. The ledger state does not immediately change.
- **Approval Request:** User 2 receives a notification/prompt to verify the payment.
- **Resolution:** Upon approval, the owed USD balances are adjusted accordingly via event sourcing. Rejections leave the ledger unchanged.

3. Technical Architecture: Event Sourcing & Sync

To support offline capabilities and multi-device sync, the application utilizes an Event Log pattern. Local mutations are queued and synced with a centralized server.

3.1 Event Log Data Model

Column Name	Type	Description
Event_ID	UUID/ULID	Unique identifier, sortable by time.
Group_ID	UUID	Associates event with a specific group.
Event_Type	String	e.g., EXPENSE_CREATED, PAYMENT_INITIATED.
Payload	JSON	The actual data of the event.
Created_At	Timestamp	When the event occurred locally.
Client_ID	String	Identifies the originating device.

3.2 Crucial Event Types

- **EXPENSE_CREATED** : Creates new ledger entries.
- **EXPENSE_UPDATED** : Reverses old USD ledger impacts and applies new ones.
- **PAYMENT_INITIATED** : Creates a PENDING settlement.
- **PAYMENT_APPROVED** : Updates settlement to APPROVED, triggering USD debt reduction.

4. Database Schema (Read Model)

The read-model database aggregates the event log into easily queryable states.

4.1 Core Entities

Table Name	Key Columns
Users	ID , PhoneNumber , Name
Friends	User1_ID , User2_ID , Status
Groups	ID , Name , CreatedBy_ID
Expenses	ID , Original_Amount , Currency , Amount_USD , FX_Rate_to_USD
Expense_Participants	Expense_ID , User_ID , Paid_Amount , Owed_Amount
Settlements	ID , Payer_ID , Payee_ID , Amount_USD , Status (PENDING/ APPROVED)

5. Balance Calculation Logic

The formula for calculating how much User A owes User B:

- Sum the net USD differences for User A relative to User B from Expense_Participants .
- Subtract the total Amount_USD from all APPROVED Settlements where User A is Payer and User B is Payee.
- The net USD result is the current debt.